- CPSC 375: High-Performance Computing

CPSC 375: High-Performance Computing

Spring 2026

- Syllabus
- Schedule
- Upload
- The Cluster Project

Homework 13

NOTE: You are not required to hand in the following exercises, but you are strongly encouraged to complete them to strengthen your understanding of the concepts covered in class.

1. Suppose you are developing a low-latency trading application where threads must update a shared `price` variable.
   A. Explain why using a standard `pthread_mutex_lock` is generally more CPU-efficient than a manual `while(locked);`.
   B. Describe what happens to a thread's CPU usage when it is blocked by a mutex.
   C. If a mutex acquisition jumps from 50–100 cycles to thousands of cycles, what is happening within the
   operating system's scheduler?

2. Suppose you have a shared linked list with 1,000,000 nodes.
   A. Describe how to protect this list using a single `pthread_mutex_t`.
   B. Describe how you would protect the list by assigning a mutex to each individual node (or head node).
   C. Which approach is more likely to cause high lock contention, and why does contention make a threaded program potentially slower than a sequential one?

3. Pthreads functions do not use `errno`; they return error codes directly. Write a C code fragment for each of the following:
   A. Prints a message if `pthread_mutex_lock` returns `EDEADLK`.
   B. Handles `EPERM` when a thread tries to unlock a mutex it does not own.
   C. Uses `pthread_mutex_trylock` and handles the `EBUSY` case.

4. High-performance applications should minimize time spent in a critical section.
   A. Explain why placing a `printf()` statement inside a locked region is considered poor practice in HPC.
   B. How can you use local variables on the stack to reduce the total time a global mutex is held during a large-scale summation?

- **Welcome: Sean**
  - LogOut


Trinity College
HARTFORD CONNECTICUT